



GitHub Copilot CLI: AI at the Point of Work

Close Every Gap Between You and the Work

Think anywhere. Steer anywhere. Learn always. Act autonomously.

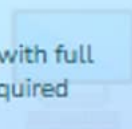
"How do I bring AI to where the work actually is?"

The IDE sees code. The terminal sees deployments, logs, CI, and production.

What unlocks when AI follows you there, and you can steer it from anywhere?

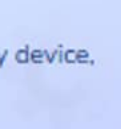
Developer

Debug Docker, CI, and production with full context — no context switching required



DevOps Engineer

Steer AI on remote servers from any device, even your phone



CLI Power User

Automate recurring workflows with session-scoped scheduled prompts



70% outside the IDE

Atlassian DevEx 2025: deployments, debugging, CI triage — all in the terminal

8 attempts → 2

Plan Mode reduces debug cycles by asking clarifying questions before code

Git-free /rewind

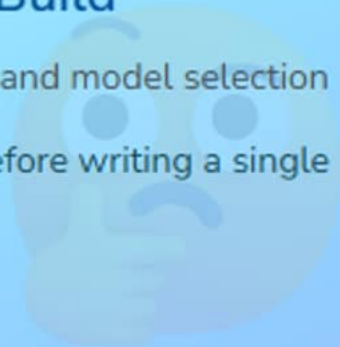
Two-mode picker: conversation only, or restore Copilot-changed files — no Git

PART 1

Think Before You Build

Plan Mode, Rubber Duck, and model selection

Ask the right questions before writing a single line of code

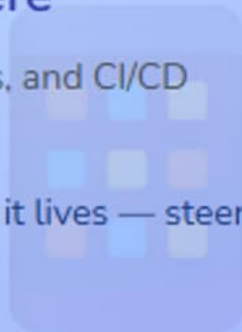


PART 2

Work From Anywhere

Remote sessions, four modes, and CI/CD integration

AI meets the problem where it lives — steer from any device

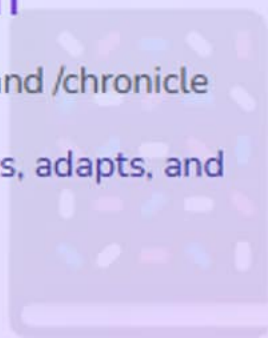


PART 3

Sessions That Learn

Auto-compaction, /rewind, and /chronicle

Durable history that recovers, adapts, and teaches the session

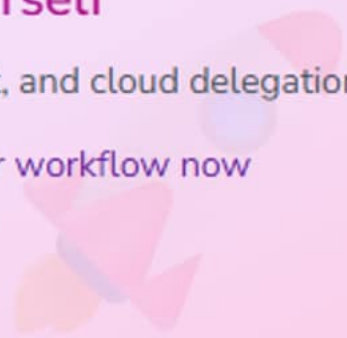


PART 4

Scale Beyond Yourself

Scheduled Prompts, /fleet, and cloud delegation

Patterns that learned your workflow now execute autonomously



Click any section to jump directly there

Think Before You Build

Gap closed: Intent. Plan Mode asks the right questions before code runs; Rubber Duck reviews what one model can miss.



Plan Mode

Clarifying Q&A first — 8 attempts → 2



Rubber Duck

Cross-family model catches what the first misses



/model

Session-scoped selection with premium multiplier

Docker debug: trial-and-error vs Plan Mode
↳ 45 min, 8 attempts → 8 min, 2 attempts

The IDE Only Sees Code

The terminal sees everything else — and that's where the work actually is

70%

of developer time spent outside the IDE

Atlassian State of Developer Experience 2025

Container logs and process state

Runtime diagnostics live in the terminal — not your source files

Network traffic and environment

Connectivity, env vars, and service health checked at the shell

File system and deployment artifacts

Storage, permissions, and build outputs live here

CI/CD pipelines and failures

Build failures, test results, and deployment validation

 What unlocks when AI follows you into the 70% — and you can steer it from anywhere?

Traditional Workflow vs. Plan Mode

From trial-and-error to collaborative strategy

✗ Traditional Workflow

- 1 Make request
Describe what you need
- 2 AI generates solution
Assumes intent — one interpretation
- 3 Review and find wrong assumptions
Mismatch surfaces in code review
- 4 Repeat until it works
Average: 8 attempts



8 attempts — one model, one interpretation, repeated guessing

✓ With Plan Mode

- 1 Make request
Describe what you need
- 2 AI asks clarifying questions
Check config, logs, or both?
- 3 Collaborate on a plan
Approve strategy before any code is written
- 4 Execute with confidence
Ambiguity resolved upfront



2 attempts — clarifying questions first, plan approved, then code

Shift+Tab to toggle Plan Mode · /goal keeps autopilot anchored to one objective

Plan Mode in Action: Docker Debugging

Root cause in 8 minutes instead of 45

```
$ copilot
> "Debug why the backend container won't start"
🤖 Copilot (Plan Mode):
  Before I start, a few questions:
  1. Check docker-compose config, logs, or both?
  2. Include environment variable analysis?
  3. Any recent changes to services or networking?
> "Start with logs, then check config if needed"

🤖 Executing plan ...
  Checked backend container logs – found connection error on port 5000
  Inspected docker-compose.yml – port mapping mismatch confirmed
✅ Found: Port 5000 mapped to 5001 – app expects 5000 internally
✅ Fix: Change service.ports from 5001:5000 to 5000:5000
```

45 min, 8 attempts → 8 min, 2 targeted checks

Rubber Duck Reviewer + Model Selection

Rubber Duck Reviewer

Cross-family model review

Claude orchestrator? Rubber Duck uses GPT-5.4 to catch blind spots

Enabled by default (v1.0.58+)

Auto-reviews after planning and significant implementations

~75% quality gap closed

Near-top-tier quality at mid-tier pricing on complex tasks

Disable: `builtInAgents.rubberDuck: false` if latency matters more

Model Selection

`/model` — session-scoped picker

Check the multiplier (1x, 2x) before choosing

`/config model` or `/model --global`

Durable default for future sessions

Never treat a catalog name as permanent — GitHub retires models

Enterprise policy may restrict replacements — verify with your admin

```
/model           # session-scoped picker
/config model    # durable default
/model --global  # same as /config model
```

Work From Anywhere

Gap closed: Geography. A phone controls a production server in 60 seconds — then modes, permissions, and CI/CD follow.



--remote

Phone steers a prod server in 60 seconds



Four Modes

Interactive, Plan, Programmatic, Remote



Sandboxes

Local MXC containment or billed cloud session

Phone → production server session

↳ Remote control via GitHub Mobile (Public Preview: 2026-04-13)

Four Modes Cover Every Workflow



Interactive

Conversational sessions with persistent context —
Shift+Tab for Plan Mode, < for Sessions sidebar



Plan Mode

AI asks clarifying questions, you approve the
strategy — then code runs. Human-driven by design



Programmatic

`copilot -p` for headless CI/CD execution — stateless,
structured output, no conversation history



Remote

`copilot --remote` generates a URL and QR code —
steer any session from GitHub.com or GitHub
Mobile



The session is the durable entity — terminals are viewports that connect and disconnect from it.

--remote: AI Meets the Problem Where It Lives

```
prod-server-3.us-east-1
```

```
ssh ops@prod-server-3.us-east-1  
$ copilot --remote
```

```
Remote session started.  
Steer this session from:  
  https://github.com/copilot/sessions/abc123  
  [QR CODE]
```

```
# From your phone, tablet, or any browser:  
# See what Copilot is doing in real time  
# Approve or deny tool permissions  
# Send steering messages  
# /resume from a different machine entirely
```

```
# Enable mid-session without restarting:  
> /remote
```



Logs stay on the server

No scp or rsync — analysis runs where the data lives, no compliance risk



Any device is a viewport

Devices are interchangeable windows into a durable AI session



tmux for survival

Combine --remote with tmux — sessions survive SSH disconnects

Sandboxes Are Containment — Not a Permission Shortcut

Local Sandbox (experimental, free)

`/sandbox enable in-session`

Persists until `/sandbox disable` — included in your Copilot seat

MXC OS containment

Lighter-weight than a VM — restricts filesystem, network, and system access

Built-in file tools honor the policy on a best-effort basis

Requires `copilot --experimental` to unlock

```
copilot --experimental
> /sandbox enable
> /sandbox status
```

Cloud Sandbox (experimental, billed)

`copilot --cloud --experimental`

Entire session in a GitHub-hosted Linux environment

Cannot combine with `-p` or `-i`

Interactive-only — no programmatic mode

Org/enterprise: disabled by default — admin must enable

Enforce no-bypass in managed config

`permissions.disableBypassPermissionsMode: true`

Programmatic Mode: Machine-Side Automation

.github/workflows/build.yml

```
- name: Analyze build failure
  if: failure()
  env:
    GITHUB_TOKEN: ${{ secrets.COPILOT_GITHUB_TOKEN }}
  run: |
    copilot -p "Analyze the build failure.
    Check recent commits and error patterns.
    Suggest specific fixes."
    --allow-tool shell(gh)
    --allow-tool shell(git)
    > failure-analysis.txt

- name: Post analysis as PR comment
  if: failure()
  run: gh pr comment --body-file failure-analysis.txt
```

⚡ 25 min → 5 min investigation

Pattern recognition across historical failures —
zero human intervention for known failure types

🔒 Scoped permissions only

--allow-tool grants specific tools needed —
never --allow-all in CI pipelines

📄 Structured output for scripts

Pipe copilot -p output to files, PR comments, or
Slack notifications

Sessions That Learn

Gap closed: Memory. S1 taught better questions, S2 taught steering anywhere — now the session learns as it works.



Auto-compaction

95% token limit → history flows forward



/rewind

Git-free picker: conversation or + files



/chronicle

History → standup, tips, better instructions

```
/chronicle standup
```

```
↳ Yesterday's work → 30-second standup report
```

Forward: Auto-compaction · Backward: /rewind

➡ Forward: Auto-compaction

At 95% token limit

Copilot compresses history transparently in the background

Decisions and patterns persist

Verbose outputs pruned; important context kept

/compact anytime for manual control — Escape to cancel

Enables virtually infinite sessions worth reconnecting to

⏮ Backward: /rewind (alias /undo)

Git-free turn picker

Not git checkout or git revert — no clean tree required

Conversation only

Rewind the chat, leave files exactly as they are

Conversation + files

Restore Copilot-changed files from that turn forward — your own edits are skipped

File changes tracked per turn across editing tools, shell commands, and sub-agents

/chronicle Turns History Into Intelligence



/chronicle standup

Generate yesterday's standup from session history in 30 seconds — not 10 minutes

Prompts you used

Files you modified

Decisions you made



/chronicle tips

AI reviews your actual usage patterns and surfaces personalized improvement suggestions

Commands you overuse

Patterns you repeat

Shortcuts you miss



/chronicle improve

Analyzes this repo's sessions and proposes specific additions to `.github/copilot-instructions.md`

Team conventions surfaced

Anti-patterns flagged

Preferences extracted

/chronicle improve: AI Shaped by How You Work

The session's emotional peak — AI that gets better the way you do

```
$ copilot
> "/chronicle improve"
📖 Reviewing 47 sessions ...
  Proposed additions to .github/copilot-instructions.md:
  1. Always check docker-compose networking before container logs
  2. Prefer async/await over promise chains in this codebase
  3. Run integration tests before unit tests – they catch more here
> "Apply suggestions 1 and 3"
✅ .github/copilot-instructions.md updated – 2 instructions added
✅ Future sessions inherit your team's proven patterns automatically
```

AI that gets better the way you do — by working

You Taught It How You Work. Now It Executes.

Started before the talk. Still firing. Watch it arrive.

```
$ copilot --experimental
```

```
> "/every 1m tell me I'm awesome"
```

🕒 **Scheduled: fires every 60 seconds**

```
[09:14:23] You're awesome!
```

```
[09:15:23] You're awesome!
```

```
[09:16:23] You're awesome! – live on screen, right now
```

✓ Manage with bare /every or /after (no arguments = list) – not /session

✓ S4: what happens when the prompt is actually useful

Session-scoped, experimental — restarts on --continue or --resume

Scale Beyond Yourself

Gap closed: Reach. S3's learned patterns become the substrate for S4's autonomous execution.



Scheduled Prompts

/every and /after — recurring agent runs



/fleet fan-out

Parallel subagents, each in their own context



Cloud delegation

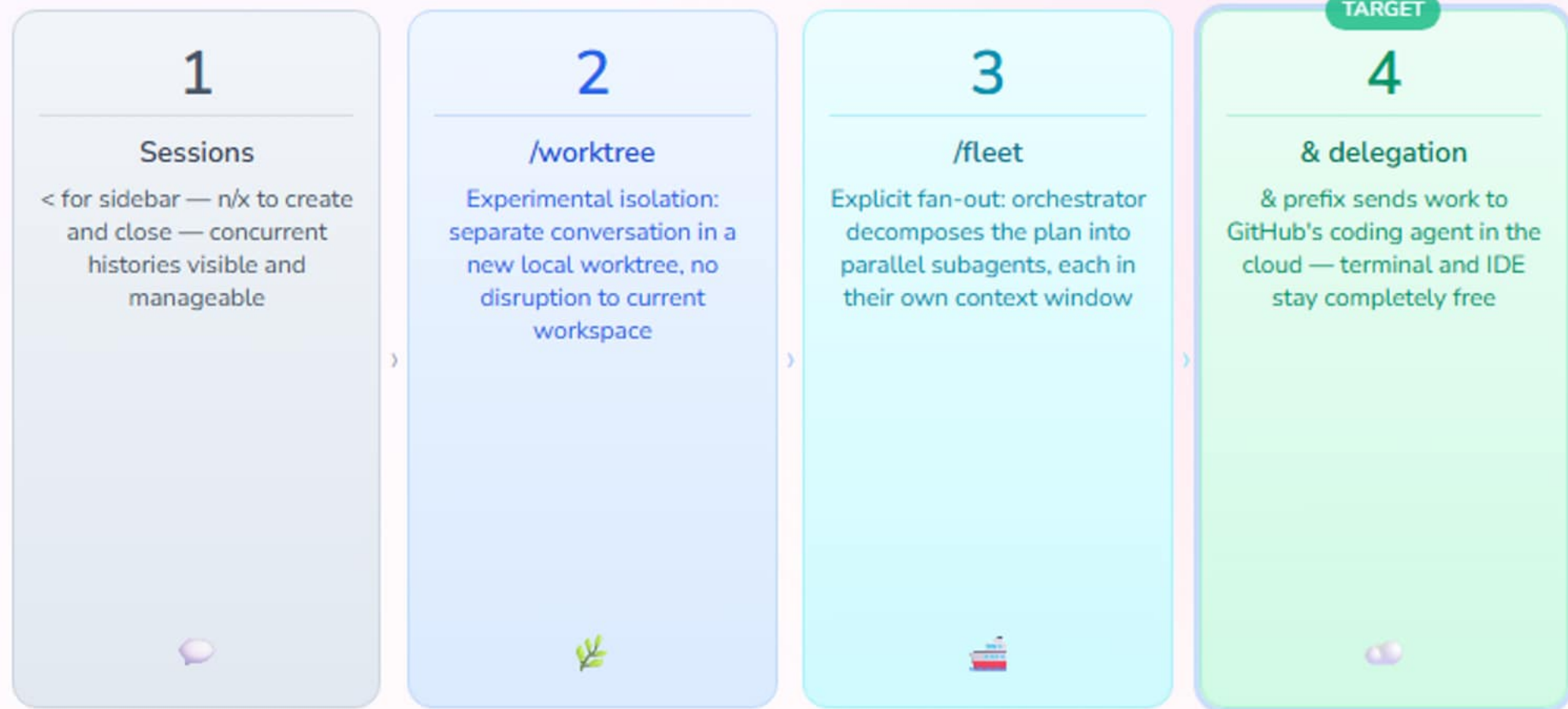
& prefix frees both terminal and IDE

Human-explicit → Automated-implicit

↳ Patterns execute autonomously – no trigger required

From Concurrent Sessions to Autonomous Execution

Each rung extends your reach without losing control



Scheduled Prompts (/every, /after) make any rung autonomous — session-scoped, experimental

Four Capabilities That Complete the Ladder



Scheduled Prompts

Bare `/every` or `/after` — not `/session`. Experimental.
No session? `cron + copilot -p`.



`/tasks + autopilot`

`/tasks` lists running subagents. `-p --plan --mode`
autopilot plans then implements.



Agent Plugins 1.0 (GA)

One package: portable skills + MCP config. Install
with `copilot plugin install`.



`/app + /ide`

`/ide` opens a file in VS Code. `/app` hands session +
folder to Copilot app (1.1.3+).



You started asking better questions. You ended with an AI system shaped by how your team works — and it executes without asking.

From Terminal Isolation to AI Everywhere

Before

Manual SSH → copy-paste output → lose context switching tools

8 trial-and-error debugging attempts before finding root cause

Session ends when you close the terminal

AI stays at your desk — even when the problem is on the server

After

Steer a production server session from your phone in 60 seconds

2 targeted attempts with Plan Mode's clarifying questions first

Sessions survive compaction, device switches, and disconnects

/chronicle converts history into standup reports and team instructions

8 → 2

debugging attempts with Plan Mode

45 → 8 min

Docker root cause with collaborative planning

60 sec

to steer a remote production server from your phone

✓ What You Can Do Today

Today

- Install GitHub Copilot CLI and run copilot in your terminal
- Use Shift+Tab to toggle Plan Mode on your next debugging session
- Run /chronicle standup after today's work session

This Week

- Start a --remote session on a staging server and steer it from your phone
- Use /rewind to recover from a wrong-turn refactor — no Git required
- Schedule /every weekday at 9am to summarize overnight PRs

This Month

- Wire copilot -p into GitHub Actions for automated build failure analysis
- Run /chronicle improve and apply suggestions to .github/copilot-instructions.md
- Delegate a long-running task with & and let the PR arrive while your IDE stays free

🔑 Key Takeaway

The session is the unit of work — think anywhere, steer anywhere, learn always, act autonomously.

 Core Documentation[About GitHub Copilot CLI](#)

Core concepts, architecture, and capabilities overview

[Use Copilot CLI](#)

Command syntax, options, and workflow patterns

[Install Copilot CLI](#)

Setup instructions for all platforms

[CLI Command Reference](#)

All commands including `/rewind`, `/every`, `/after`, and `/fleet`

 Features and How-Tos[Cloud and Local Sandboxes](#)

Experimental `/sandbox enable` and `copilot --cloud --experimental`

[Steering a Session Remotely](#)

Phone and browser control for `--remote` sessions

[Using `/chronicle`](#)

Standup reports, personalized tips, and instruction improvements

[Scheduling Prompts](#)

`/every` and `/after` — experimental session-scoped automation

[Running Tasks With `/fleet`](#)

Explicit fan-out across parallel subagents with context isolation

[Finding and Installing Plugins](#)

Agent Plugins 1.0 (GA) — consumer install guide

 Recent Updates

[Remote Control CLI Sessions \(Public Preview\)](#)



GitHub Copilot CLI

AI at the Point of Work

8 → 2

debugging attempts with Plan Mode

60 sec

to steer a production server from your
phone

4 gaps

closed: Intent, Geography, Memory, Scale

Which gap does your team hit most — intent, geography, memory, or scale?

Interactive Mode — Agent Environment

<code>/init</code>	Initialize Copilot instructions for this repo
<code>/agent</code>	Browse and select from available agents
<code>/skills</code>	Manage skills for enhanced capabilities
<code>/mcp</code>	Manage MCP server configuration
<code>/plugin</code>	Manage plugins and plugin marketplaces
<code>/instructions</code>	View & toggle custom instruction files
<code>/env</code>	Show loaded instructions, MCPs, skills, agents

What this gives you

Everything that shapes the agent's behavior in this session — instructions, custom agents, skills, MCP servers, and plugins.

Start here

`/init` once per repo, then `/env` any time you wonder "what does the agent actually know right now?"

Interactive Mode — Models & Subagents

<code>/model</code>	Select AI model to use
<code>/autopilot</code>	Toggle autopilot mode
<code>/fleet</code>	Parallel subagent execution
<code>/delegate</code>	Send session to GitHub → create a PR
<code>/tasks</code>	View & manage tasks (subagents, shell)
<code>/sidekicks</code>	View running sidekick agents

Three execution gears

- Interactive — you approve each step
- Autopilot — agent runs until done
- Fleet — multiple subagents in parallel

Hand-off vs parallelize

`/delegate` ships the work to GitHub coding agent;
`/fleet` stays local and fans out subagents.

Interactive Mode — Code

<code>/plan</code>	Implementation plan before coding
<code>/diff</code>	Review the changes made in current di
<code>/pr</code>	Operate on pull requests for this branch
<code>/review</code>	Run code review agent on changes
<code>/ide</code>	Connect to an IDE workspace
<code>/lsp</code>	Manage language server configuration
<code>/terminal-setup</code>	Configure shift+enter for multiline

Plan → Code → Review

`/plan` drafts the approach, the agent codes, then `/diff` and `/review` let you inspect & refine before opening a PR.

IDE bridge

`/ide` pairs the CLI with an open editor — file edits open as native diffs and selections flow back to Copilot.

Interactive Mode — Session

<code>/resume</code>	Switch sessions (id / task id / name)
<code>/session</code>	View & manage sessions
<code>/rename</code>	Rename or auto-name current session
<code>/context</code>	Show context window token usage
<code>/usage</code>	Session usage metrics & stats
<code>/compact</code>	Summarize history to free context
<code>/memory</code>	Toggle cross-session fact recall
<code>/rewind</code>	Undo last turn + revert file changes
<code>/share</code>	Export to markdown / HTML / gist
<code>/copy</code>	Copy last response to clipboard
<code>/remote</code>	Toggle control from GitHub web / mobile

Context hygiene

Watch `/context` . When it's > 70% full, run `/compact` with focus instructions instead of starting over.

Sessions are persistent

All sessions live in `~/.copilot/session-store.db` . `/resume` picks any one up — even days later, by name or ID prefix.

Interactive Mode — Permissions

<code>/allow-all</code>	Enable all permissions (tools, paths, URLs)
<code>/add-dir</code>	Whitelist a directory for file access
<code>/list-dirs</code>	Show all allowed directories
<code>/cwd</code>	Change or show working directory
<code>/reset-allowed-tools</code>	Clear the list of allowed tools

Three permission surfaces

- Tools — what the agent can invoke
- Paths — what files it can touch
- URLs — what domains it can reach

Denial always wins

Deny rules override `/allow-all`. Safe pattern: allow broadly, deny specifically (e.g. `shell(git push)`).

Interactive Mode — Help & More

<code>/research</code>	Deep research via GitHub + web
<code>/chronicle</code>	Session history insights
<code>/changelog</code>	CLI version history (+ AI summary)
<code>/new</code>	Start a new conversation
<code>/clear</code>	Abandon session and start fresh
<code>/keep-alive</code>	Prevent system sleep during work
<code>/voice</code>	Dictation via Foundry Local
<code>/theme</code>	View or set color mode
<code>/streamer-mode</code>	Hide model names / quota (screen share)
<code>/experimental</code>	Enable / disable experimental features
<code>/feedback</code>	Provide feedback about the CLI
<code>/help</code>	Show help for interactive commands

Long-running work

`/research` spins up a deep-dive task; pair with `/keep-alive` so your laptop doesn't sleep mid-run.

Demo & screen-share kit

`/streamer-mode` hides quota and preview model names; `/theme` swaps light/dark to match your slides.

Non-Interactive Mode — Scriptable AI

One-shot prompt, exits after response

```
copilot -p "Fix the failing test in auth.test.ts" --allow-all
```

Quiet output – only agent response, no stats (great for piping)

```
copilot -p "Summarize this PR" --silent | pbcopy
```

Autopilot – continues autonomously until done

```
copilot -p "Migrate all API routes to use the new auth middleware" --allow-all --autopilot
```

Share session output as a GitHub gist

```
copilot -p "Audit this repo for security issues" --allow-all --share-gist
```

Structured JSON output for CI pipelines

```
copilot -p "Run tests and report failures" --allow-all --output-format json
```

Permissions — Granular Control

```
# Allow specific tool categories
copilot --allow-tool=write --allow-tool=shell

# Allow all git except push
copilot --allow-tool='shell(git:*)' \
--deny-tool='shell(git push)'

# Restrict to specific directories
copilot --add-dir ~/workspace --add-dir /tmp

# Allow GitHub API access
copilot --allow-url=github.com

# Enable everything – tools, paths, URLs
copilot --yolo

# Same as:
copilot --allow-all-tools \
--allow-all-paths \
--allow-all-urls

--allow-all shorthand for all three
--yolo alias for --allow-all

# Required for non-interactive (-p) mode
copilot -p "... " --allow-all-tools
```

`copilot init` — Bootstrap Your Repo

Analyzing codebase structure ...

Detecting build system: npm

Detecting test framework: jest

Detecting language: TypeScript

✓ Created `.github/copilot-instructions.md`

What it generates

Analyzes your repo with read-only tools and writes `.github/copilot-instructions.md` with:

- Build & test commands
- Coding conventions
- Project structure overview
- Technology stack details

Why it matters

Every future session in this repo automatically loads these instructions — no need to re-explain your project to Copilot.

`copilot login` — Authentication

```
# Interactive browser flow (default)
copilot login

# GitHub Enterprise Cloud (data residency)
copilot login --host https://example.ghe.com

# Headless / CI - use env var instead of login
COPILOT_GITHUB_TOKEN=github_pat_... copilot

# Token precedence order:
COPILOT_GITHUB_TOKEN
GH_TOKEN
GITHUB_TOKEN
```

Supported token types

- Fine-grained PATs (v2) with *Copilot Requests* permission
- OAuth tokens from GitHub Copilot CLI app
- OAuth tokens from GitHub CLI (gh) app

✗ Classic PATs (ghp_...) are not supported

Token storage

Stored in system credential store. Falls back to plain text in `~/.copilot/` if no credential store is available.

`copilot plugin` — Extend with Plugins

```
# Browse the default marketplace
copilot plugin marketplace browse copilot-plugin

# Install from GitHub repository
copilot plugin install github/my-plugin

# Install from local directory
copilot plugin install ./my-local-plugin

# List installed plugins
copilot plugin list

# Update / uninstall
copilot plugin update my-plugin
copilot plugin uninstall my-plugin
```

What plugins can add

- Additional skills & agents
- Git hooks
- MCP servers
- LSP servers

Built-in marketplaces

```
copilot-plugins github/copilot-plugins
awesome-copilot github/awesome-copilot
```

copilot help config — Key Settings

Behavior

model – AI model (gpt-5.4, claude-sonnet-4.6, ...)
autoUpdate – auto-download CLI updates (default: true)
memory – agentic cross-session fact recall (default: true)
keepAlive – prevent system sleep (off / on / busy)
experimental – enable experimental features
renderMarkdown – render markdown in terminal
respectGitignore – hide gitignored files from @ picker
includeCoAuthoredBy – add Co-authored-by to commits
compactPaste – collapse large pastes (>10 lines)
streamerMode – hide model names & quota (for screen sharing)

Terminal

mouse – enable mouse support in alt screen
beep – beep when attention required

Security

allowedUrls – pre-approved URL/domain list (supports *.domain)
deniedUrls – blocked domains (takes precedence)
trustedFolders – folders with persisted read/exec approval

Available models

claude-sonnet-4.6 claude-sonnet-4.5 claude-haiku-4.5
claude-opus-4.8 claude-opus-4.7 claude-opus-4.6
claude-opus-4.6-fast claude-opus-4.5
gpt-5.5 gpt-5.4 gpt-5.2
gpt-5.3-codex gpt-5.2-codex
gpt-5.4-mini gpt-5-mini

Teams

companyAnnouncements – messages shown in banner on startup

copilot help environment — Key Env Vars

Authentication

`COPILOT_GITHUB_TOKEN` – PAT / OAuth token (highest precedence)

`GH_TOKEN` – GitHub CLI token (second)

`GITHUB_TOKEN` – CI token (third)

`GH_HOST` – GitHub Enterprise hostname

`COPILOT_GH_HOST` – Copilot-only host (overrides `GH_HOST`)

Behavior

`COPILOT_ALLOW_ALL=true` – skip all confirmations

`COPILOT_MODEL` – override AI model

`COPILOT_AUTO_UPDATE=false` – disable auto-update

`COPILOT_HOME` – override `~/.copilot` directory

`COPILOT_OFFLINE=true` – no network (needs BYOK)

`COPILOT_CUSTOM_INSTRUCTIONS_DIRS` – extra `AGENTS.md` dirs

BYOK – Bring Your Own Key

`COPILOT_PROVIDER_BASE_URL` – custom API endpoint (activates BYOK)

`COPILOT_PROVIDER_TYPE` – openai / azure / anthropic

`COPILOT_PROVIDER_API_KEY` – API key

`COPILOT_PROVIDER_BEARER_TOKEN` – bearer token (takes precedence over API key)

`COPILOT_PROVIDER_WIRE_API` – completions / responses (use responses for GPT-5)

`COPILOT_PROVIDER_AZURE_API_VERSION` – Azure API version

Proxy & Output

`HTTP_PROXY` / `HTTPS_PROXY` – outbound proxy URL

`NO_PROXY` – bypass list (comma-separated)

`NO_COLOR` – disable color output

`PLAIN_DIFF=true` – disable rich diff

BYOK — Bring Your Own Key

```
# Ollama (local, no auth required)
COPILOT_PROVIDER_BASE_URL=http://localhost:11434/v1 \
COPILOT_MODEL=deepseek-coder-v2:16b \
copilot

# Custom OpenAI-compatible endpoint
COPILOT_PROVIDER_BASE_URL=https://my-api.example.com/v1 \
COPILOT_PROVIDER_API_KEY=sk- ... \
COPILOT_MODEL=gpt-4 \
copilot

# Azure OpenAI with deployment name
COPILOT_PROVIDER_TYPE=azure \
COPILOT_PROVIDER_BASE_URL=https://my.openai.azure.com \
COPILOT_PROVIDER_MODEL_ID=gpt-4 \
COPILOT_PROVIDER_WIRE_MODEL=my-gpt4-deployment \
copilot
```

Provider types

openai – default; covers Ollama, vLLM, Foundry Local
azure – Azure OpenAI (host URL only)
anthropic – Anthropic API

When to use

- Air-gapped or offline environments
- Enterprise contracts with specific model providers
- Local development with Ollama / Foundry Local
- Cost control with self-hosted models

`/fleet` — Parallel Subagent Execution

*Refactor auth module, add unit tests,
update API docs, run the linter.*

`/fleet`

✓ Plan — 4 subtasks identified

[subagent-1] Refactor auth module

[subagent-2] Write unit tests

[subagent-3] Update API documentation

[subagent-4] Run linter across repo

Running in parallel...

✓ [subagent-4] Linter: 3 warnings fixed

✓ [subagent-1] Auth refactored

✓ [subagent-3] 12 endpoints updated

✓ [subagent-2] 24 tests added, passing

✓ All subtasks complete

How it works

- Main agent acts as orchestrator
- Decomposes prompt into independent subtasks
- Runs in parallel, each with its own context
- Manages dependencies; merges results

Best used for

- Multi-file refactors + test generation
- Audits + fixes across modules
- Large dependency updates in monorepos

Power combos

Shift+Tab → plan → Accept + autopilot + `/fleet`
"Use GPT-5.3-Codex to refactor... "
"Use @test-writer to create tests... "

Where the Magic Lives — `~/.copilot/`

How Copilot remembers everything — in-session and between sessions

<code>agents/</code>	custom agents
<code>skills/</code>	skill packs
<code>ide/</code>	IDE bridge
<code>logs/</code> <code>session-state/</code>	in-session state
<code>pkg/</code>	CLI & auto-update
<code>restart/</code>	restart staging
<code>config.json</code>	settings
<code>permissions-*.json</code>	allow/deny rules
<code>session-store.db</code>	past sessions (SQLite)
<code>command-history-*.json</code>	prompt history

In-session

- `session-state/` — context, tasks, subagent state
- `ide/` — live editor bridge
- `logs/` — execution logs

Between-session

- `session-store.db` — all conversations, resumable
- `command-history-*.json` — up-arrow recall
- `permissions-*.json` — persistent approvals

Portable capabilities

- `agents/` & `skills/` follow you everywhere
- Override with `$COPILOT_HOME`